



**FACULTY OF ELECTRONICS AND COMPUTER
TECHNOLOGY AND ENGINEERING**

SEM II - 2025/26

BERR 2243

DATABASE AND CLOUD SYSTEM

EXERCISE 4 REPORT

LECTURER NAME:

PROFESOR MADYA DR SOO YEW GUAN

No.	Student Name	Matrix No.
1	IKMAL IMAN BIN MOHD YUSOF	B122410306
2	MUHAMMAD ARIF SYAFIE BIN AGUS SALEH	B022210116
3	MUHAMMAD HAIKAL BIN ZULKIFLI	B122410348

Table of Content

1. Objective.....	1
2. Methodology & Setup Procedure.....	1
a. Initializing the EMQX Broker.....	1
b. Configuring the Publisher (Node-RED).....	2
c. Configuring the Subscriber.....	5
3. Results & Observations.....	6
a. Broker Traffic Verification.....	6
b. Direct Topic Subscription.....	6
c. Single-Level Wildcard (+) Observations.....	7
d. Multi-Level Wildcard (#) Observations.....	8
e. Wildcard Syntax Limitation.....	8
4. Discussion & Architectural Analysis.....	9
Question 1: The Role of the Broker.....	9
Question 2: Publish vs. Subscribe.....	10
Question 3: Client vs. Server.....	11
5. Conclusion.....	12

1. Objective

The primary objectives of this laboratory exercise are:

- To deploy an enterprise-grade MQTT broker (EMQX) locally via Docker.
- To configure Node-RED to act as an automated MQTT publishing client.
- To use Postman as an MQTT subscriber to capture data.
- To explore and understand MQTT topic hierarchies using multi-level and single-level wildcards.

2. Methodology & Setup Procedure

a. Initializing the EMQX Broker

- Deployment and Container Configuration:** The enterprise-grade MQTT broker, EMQX, was deployed locally using Docker Desktop, utilising the `emqx:latest` image. Essential network ports were mapped to allow communication, specifically exposing port 1883 for standard MQTT traffic and port 18083 for the web-based management dashboard. Additionally, to ensure data persistence, local host directories were bound to the container's internal paths (D:\emqx\data mapped to `/opt/emqx/data`, the Mnesia database and D:\emqx\log mapped to `/opt/emqx/log` for operational logs).

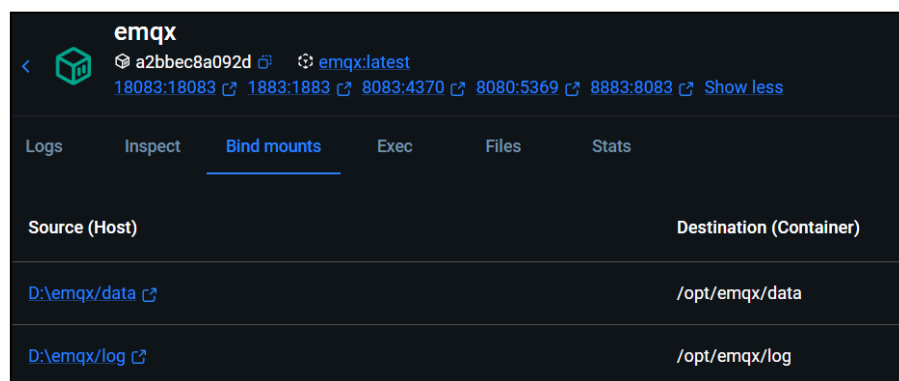


Fig. 2a. 1. EMQX Docker Configuration

- Broker Verification:** Once the container was running, the broker's successful initialisation was verified by accessing the EMQX Cluster Overview dashboard via a web browser at `http://localhost:18083`. This confirmed that the server was active, successfully monitoring connections, and ready to manage MQTT topics.

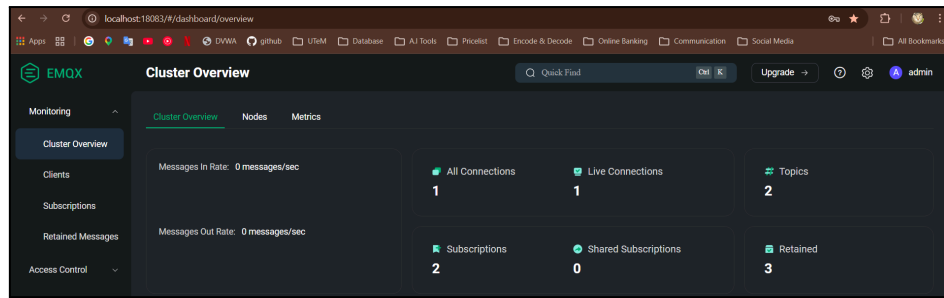


Fig. 2a. 2. EMQX Dashboard

b. Configuring the Publisher (Node-RED)

To simulate IoT edge devices sending telemetry data, Node-RED was configured as an automated MQTT publishing client. To support the exploration of topic hierarchies, three separate publishing flows were created.

- i. **Configuring the Inject Nodes:** Three distinct "inject" nodes were added to the workspace to act as data generators. Each node was configured to output a JSON payload representing sensor readings, and the repeat interval was set to inject automatically every 5 seconds.

- **Lab 1 Temperature:** Configured to send a JSON payload containing a simulated temperature of 25.5°C from a DHT11 sensor.

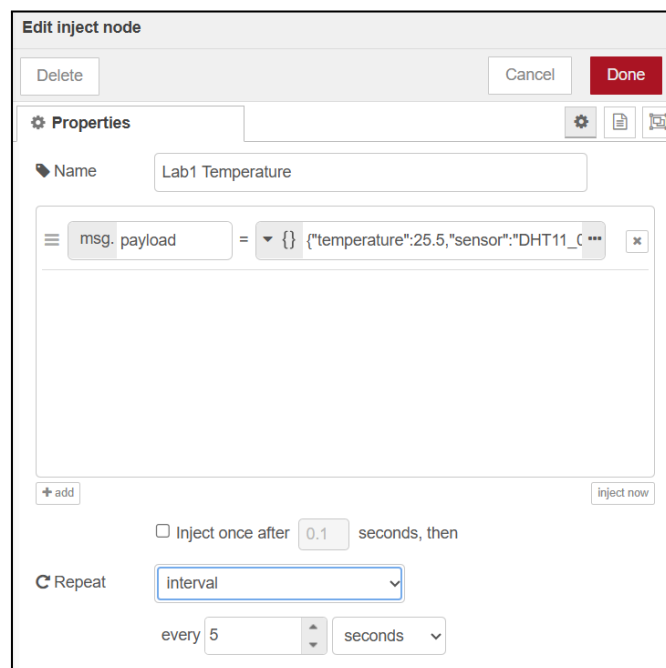


Fig. 2b. 1. The 1st Inject Node configuration

- **Lab 1 Humidity:** Configured to send a JSON payload containing a simulated humidity of 15.6% from a DHT11 sensor.

The screenshot shows the 'Edit inject node' dialog box. At the top, there are 'Delete', 'Cancel', and 'Done' buttons. Below is a 'Properties' section with a 'Name' field containing 'Lab1 Humidity'. The main configuration area shows 'msg. payload' set to a JSON object: `{ "Humidity": 15.6, "Sensor": "DHT11" }`. At the bottom, there are options for injection frequency: 'Inject once after 0.1 seconds, then' (unchecked) and 'Repeat interval' set to 'every 5 seconds'.

Fig. 2b. 2. The 2nd Inject Node configuration

- **Lab 2 Temperature:** Configured to send a JSON payload containing a simulated temperature of 23.5°C from a DHT11 sensor.

The screenshot shows the 'Edit inject node' dialog box for 'Lab2 Temperature'. The 'Name' field contains 'Lab2 Temperature'. The 'msg. payload' is set to a JSON object: `{ "temperature": 23.5, "sensor": "DHT11" }`. The injection frequency settings at the bottom are identical to the previous configuration: 'Inject once after 0.1 seconds, then' (unchecked) and 'Repeat interval' set to 'every 5 seconds'.

Fig. 2b. 3. The 3rd Inject Node configuration

- ii. **Configuring the MQTT Out Nodes:** Three 'mqtt out' nodes were added and wired to their respective inject nodes. Each node was configured to connect to the local Docker EMQX broker by setting the server to `host.docker.internal:1883`. They were then assigned distinct topic paths to organise the data:

- The first node was routed to the topic campus/lab1/temperature.

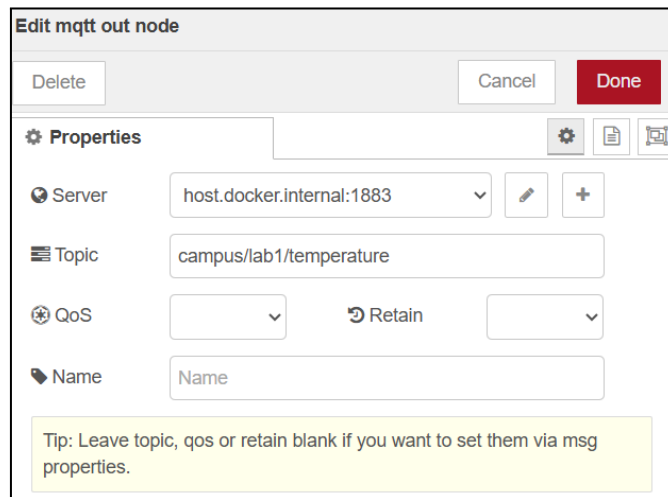


Fig. 2b. 4. 1st MQTT Out node configuration

- The second node was routed to the topic campus/lab1/humidity.

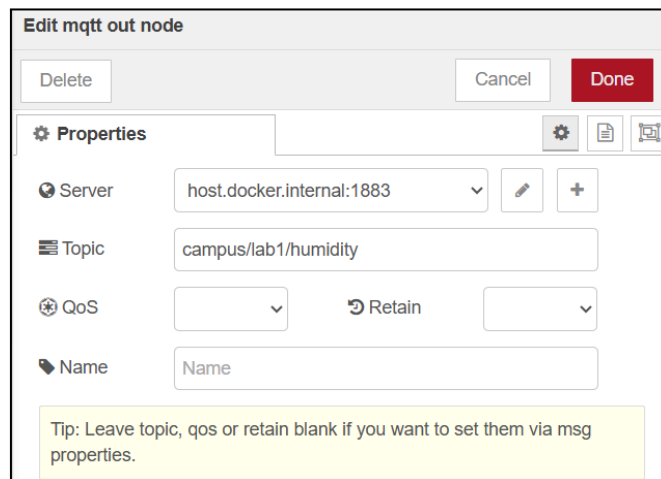


Fig. 2b. 5. 2nd MQTT Out node configuration

- The third node was routed to the topic campus/lab 2/temperature.

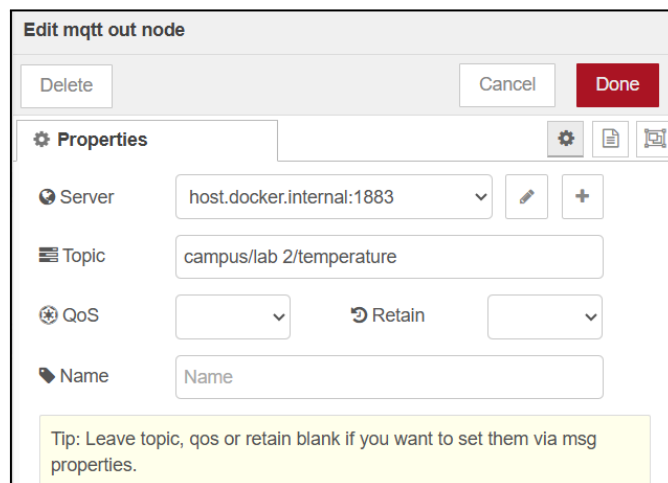


Fig. 2b. 6. 3rd MQTT Out node configuration

- iii. **Deployment and Verification:** Once all nodes were properly configured and wired, the flow was deployed. The visual indicators under the MQTT nodes in the Node-RED workspace turned green, displaying "connected", verifying that the clients successfully established a connection with the local EMQX broker and began silently publishing payloads every 5 seconds.

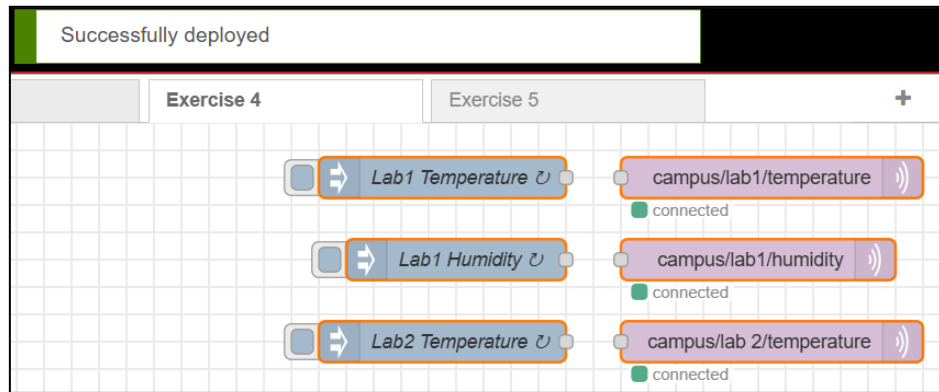


Fig. 2b.11. Node-RED Deployment

c. Configuring the Subscriber

To verify that the published telemetry data was successfully routed through the broker, Postman was utilised as an independent MQTT subscribing client.

- i. **Client Initialisation:** A new MQTT request interface was created within Postman, transitioning from the standard HTTP REST protocol to the MQTT messaging protocol.
- ii. **Broker Connection:** The client was configured to target the local EMQX broker by entering the URL localhost:1883. Upon clicking 'Connect', a green "Connected" status indicator verified that a persistent connection was successfully established with the broker.
- iii. **Topic Configuration:** In preparation for both direct data reception and advanced topic hierarchy experiments, the 'Topics' tab was populated with multiple subscription targets. This included the specific direct paths (e.g., campus/lab1/temperature) as well as wildcard expressions (e.g., campus/# and campus/+/temperature). Toggle switches were used to isolate specific topics during the observation phase.

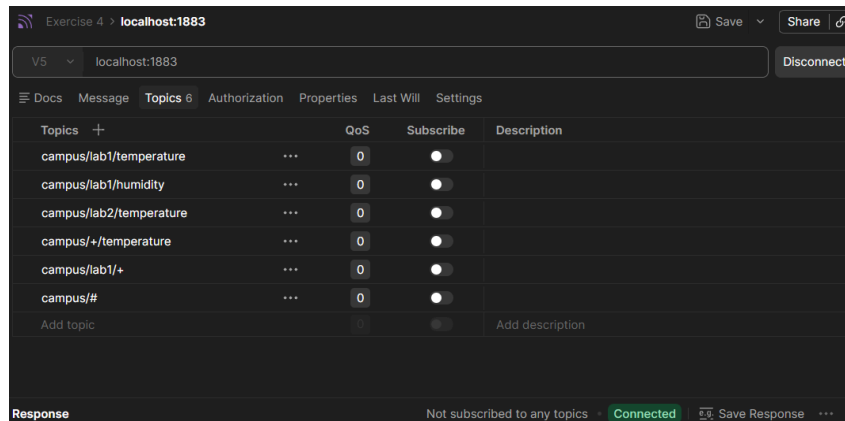


Fig. 2c. 1. Postman Configuration

3. Results & Observations

a. Broker Traffic Verification

Upon deploying the Node-RED publishers, the EMQX Cluster Overview dashboard was monitored. The dashboard successfully registered active data streams, displaying continuous metrics for both "Messages In Rate" and "Messages Out Rate". This confirmed that the Dockerised broker was actively receiving the published payloads and successfully sending them to the subscribed clients.

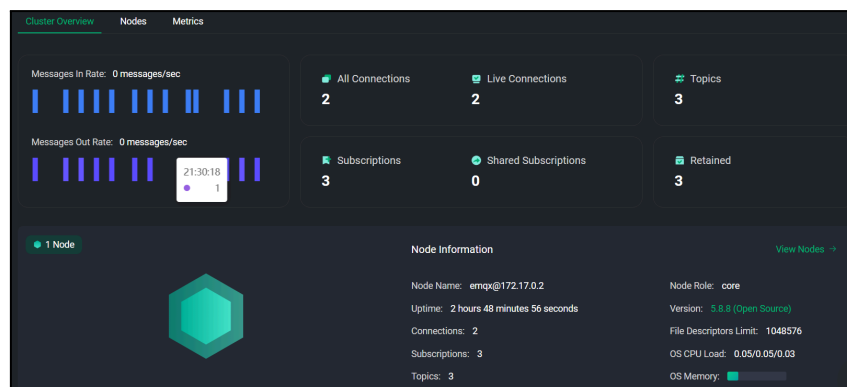


Fig. 3a. 1. EMQX Cluster Overview

b. Direct Topic Subscription

To establish a baseline, the Postman client was subscribed to topics: 'campus/lab1/temperature', 'campus/lab1/humidity', and 'campus/lab2/temperature'. The exact JSON payloads originating from all the inject nodes were received, arriving consistently at 5-second intervals, as shown by the observation.

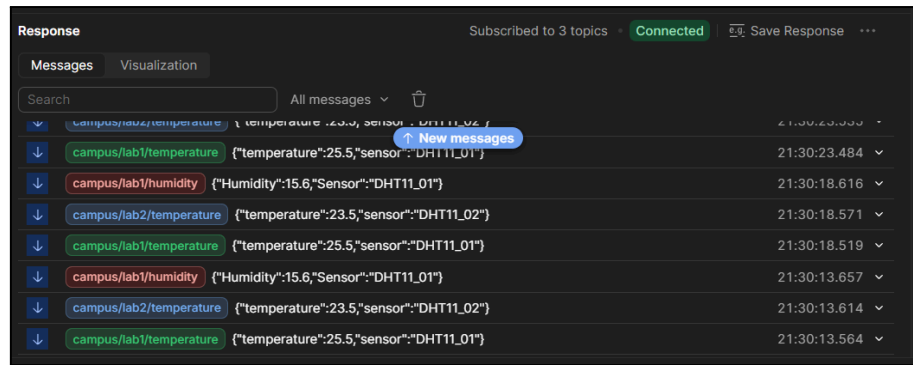


Fig. 3b. 1. Response for Direct Topic Subscription

c. Single-Level Wildcard (+) Observations

The single-level wildcard (+) substitutes exactly one level in the topic hierarchy.

- **Test 1 (campus/+/temperature):** Subscribing to this topic successfully captured temperature data from both Lab 1 and Lab 2. The humidity data was successfully filtered out because the last level strictly demanded "temperature".

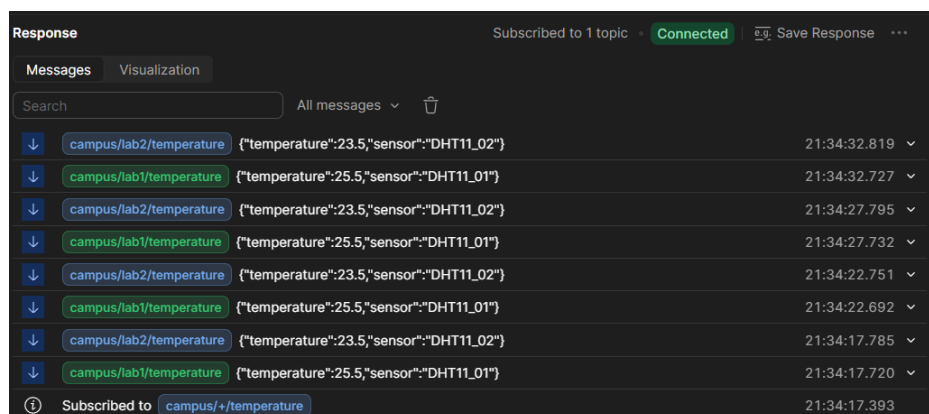


Fig. 3c. 1. Response for (campus/+/temperature) topic

- **Test 2 (campus/lab1/+):** Subscribing to this topic successfully captured both temperature and humidity data, but only from lab1. It ignored lab2 completely.

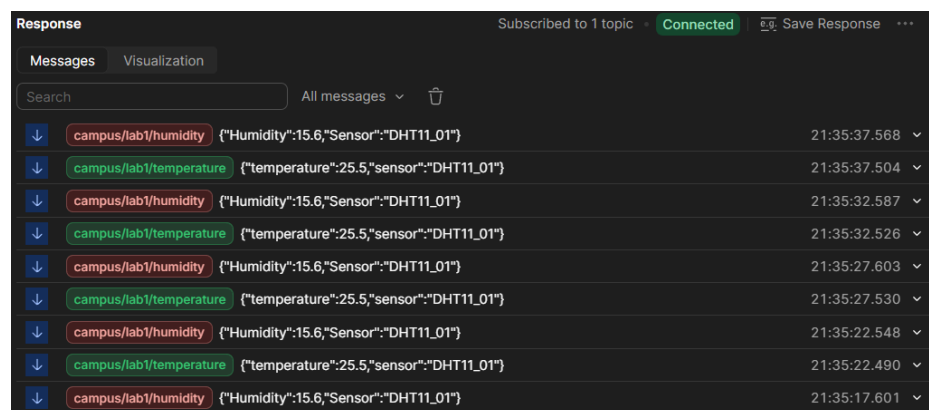


Fig. 3c. 2. Response for (campus/lab1/+) topic

d. Multi-Level Wildcard (#) Observations

The multi-level wildcard (#) matches any number of levels from that point onwards in the hierarchy.

- **Test 1 (campus/#):** This acted as a root catch-all. The client successfully received all three telemetry streams simultaneously (Lab 1 Temp, Lab 1 Humidity, and Lab 2 Temp).

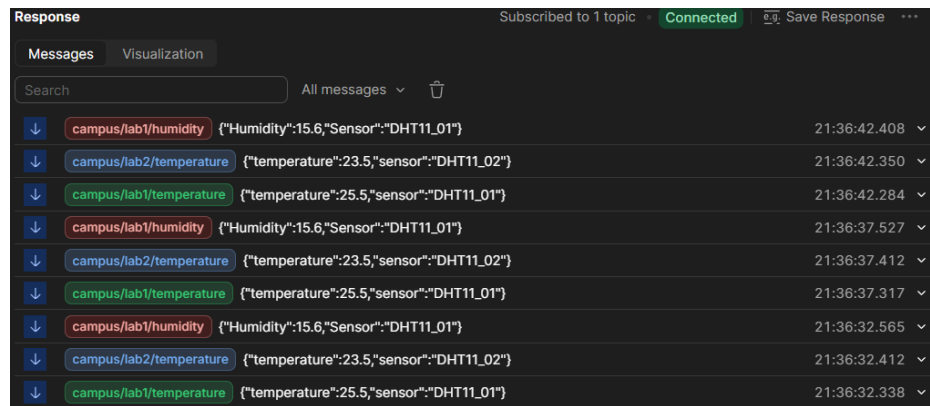


Fig. 3d. 1. Response for (campus/#) topic

- **Test 2 (campus/lab1/#):** This restricted the catch-all to a specific branch. The client successfully received all available sensor data (both temperature and humidity), specifically from Lab 1.

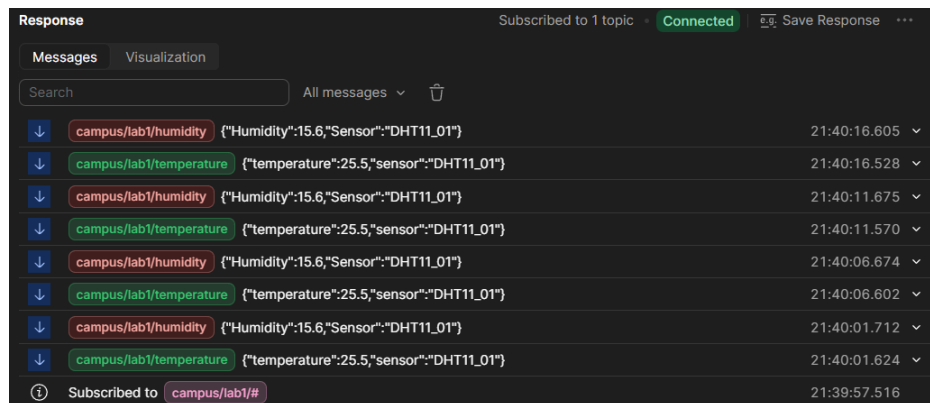


Fig. 3d. 2. Response for (campus/lab1/#) topic

e. Wildcard Syntax Limitation

An attempt was made to subscribe to the topic (campus/#!/temperature).

- **Observation:** No data was received, and the subscription failed to yield results.
- **Analysis:** This occurs because it violates the MQTT protocol rules. The multi-level wildcard (#) captures the entire remaining length of the topic tree.

Therefore, it must always be placed as the exact last character in a topic string. Adding specific levels after a '#' wildcard results in an invalid topic formulation.

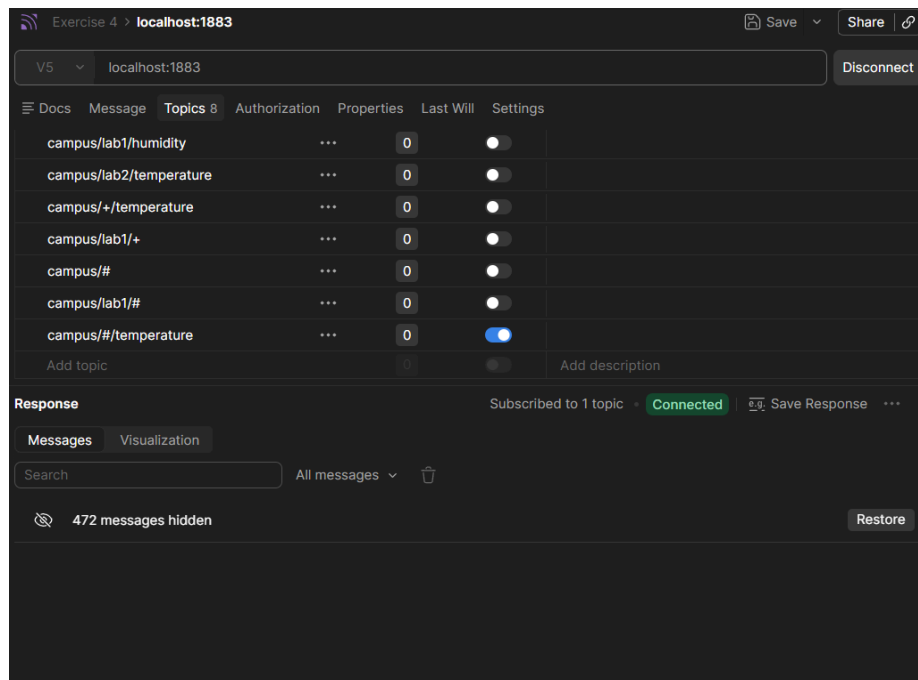


Fig. 3e. 1. Response for (campus/#/temperature) topic

4. Discussion & Architectural Analysis

Question 1: The Role of the Broker.

In Week 3, you sent data directly from Postman (client) to Node-RED (server) using HTTP. Today, both Node-RED and Postman are connected to the EMQX Broker instead. Explain the architectural role of the MQTT broker. What is the advantage of MQTT compared to the HTTP protocol?

In HTTP, communication is a direct line. Postman had to connect directly to Node-RED, meaning both had to be online at the same time and know each other's exact address.

In MQTT, the EMQX Broker acts as a central middleman. Devices never talk directly to each other. Instead, the publisher (Node-RED) pins a message to the broker, and the subscriber (Postman) reads it from the broker. This separates the devices so they don't need to know each other's IP addresses or even be online at the same time.

The Advantages of MQTT compared to HTTP:

- **Push instead of Ask (Polling):** In HTTP, a client has to constantly ask the server. This wastes battery and internet data. In MQTT, the client just waits. When new data arrives, the broker instantly pushes it to the client.
- **Lighter and Faster:** HTTP sends a lot of extra background text (headers) with every message. MQTT is designed for small IoT sensors, so it strips away the heavy stuff and sends pure, lightweight data.
- **One-to-Many Sharing:** With HTTP, sending data to 5 different devices requires 5 separate connections. With MQTT, Node-RED only sends the data once to the broker, and the broker does the hard work of copying and delivering it to all 5 devices at the same time.

Question 2: Publish vs. Subscribe

Define the specific roles of a "Publisher" and a "Subscriber" in the MQTT protocol. Can a single IoT device (like an ESP32) be both a publisher and a subscriber simultaneously? Provide a real-world example of why a device would need to do both.

The Specific Roles:

- **The Publisher (The Sender):** Its only job is to generate data (like reading a temperature sensor) and send that data to the MQTT broker under a specific topic. The publisher just drops the message off to the broker and does not care who is reading it.
- **The Subscriber (The Listener):** Its function is to register its interest in a specific topic with the MQTT broker. Once registered, it stays in standby mode, ready to receive incoming data pushed by the broker.

A single IoT device, such as an ESP32, can act as both a publisher and a subscriber simultaneously. This dual functionality is strictly required for establishing two-way communication in interactive systems. In a smart air conditioning system, the internal ESP32 microcontroller performs both roles concurrently:

- **Acting as a Publisher:** The device continuously reads the room's temperature from its sensors and publishes this data to the MQTT broker. This enables the user's mobile application to display real-time environmental updates.
- **Acting as a Subscriber:** Simultaneously, the device subscribes to a specific command topic. When a user sends a command via the app, such as power off or set the temperature to 16°C, the ESP32 instantly receives this instruction from the broker and adjusts its motor settings as required.

Question 3: Client vs. Server

In an HTTP REST API, the server must be running and listening before the client can send a POST request. In MQTT, what happens if Postman (the subscribing client) disconnects for 10 minutes while Node-RED (the publishing client) continues to send data? How does the broker handle this?

In HTTP, communication is strictly point-to-point. If the receiving server is offline, the sender's POST request immediately fails, and the data is lost.

In MQTT, the broker sits in the middle and acts like a smart post office. If Postman (the subscriber) disconnects for 10 minutes while Node-RED (the publisher) keeps sending data, the broker handles it based on how the connection was set up:

- **Clean Session:** If Postman connected with a standard, temporary session, the broker assumes Postman doesn't care about missed data. The broker will throw away the messages sent during those 10 minutes. When Postman reconnects, it starts fresh and only receives new messages moving forward.
- **Persistent Session:** If Postman registered a persistent (saved) session before disconnecting, the broker acts like a mailbox. It saves all the messages published by Node-RED during that 10-minute offline period. The moment Postman reconnects, the broker automatically delivers all the missed messages at once.
- **Retained Message:** If Node-RED is set to "retain" its messages, the broker will pin the absolute latest message to the notice board. Even if Postman was offline and missed the last 10 minutes, the second it reconnects, it will instantly receive that pinned message so it immediately knows the newest status.

5. Conclusion

In conclusion, this lab exercise successfully showed how an MQTT system works in real life. By setting up an EMQX broker, we saw how it acts as a middleman to separate the sender and receiver. Node-RED worked perfectly as the sender (publisher) to generate sensor data, while Postman acted as the listener (subscriber) to receive it. This setup proved that MQTT's "publish and subscribe" method is much smoother and more efficient than the traditional HTTP method.

We also tested the + and # wildcards, which showed how easy it is to filter and receive multiple data streams at the same time. Overall, this exercise helped us understand that MQTT is a lightweight and reliable tool, making it the perfect choice for connecting smart IoT devices and monitoring data in real time.